

Real-Time Camera-Free Deformation Reconstruction via Flexible Sensor Array and Cage-based deformation model

Abstract—We present a camera-free, real-time deformation reconstruction system for soft objects that uses a flexible piezoresistive sensor array and a learned cage-based Gaussian deformation model. Our approach combines innovations in sensor design, dynamic scene representation, and graph neural networks to achieve high-fidelity shape tracking without external cameras. First, we obtain a 4D Gaussian Splatting representation of object deformations from multi-view RGB images, yielding per-frame Gaussian primitives that capture fine surface details. We then design a flexible 10×10 piezoresistive sensor array (3×3 cm) that provides sparse pressure-induced resistance readings on the object’s surface in real time. To map these sparse signals to a dense deformation, we introduce a cage-based deformation framework: a graph attention network (GNN) predicts the motion of a set of sparse cage control points, and their displacements are propagated to millions of Gaussians to reconstruct the full high-resolution shape. This cage-based strategy decouples low-dimensional control from high-resolution representation, enabling efficient inference and zero-shot generalization to new surfaces. We demonstrate fine-grained tracking and rendering of three representative deformation types – bending, twisting, and pressing – achieving 30 FPS real-time rendering of photorealistic RGB images. Our results indicate that the proposed sensor-driven Gaussian deformation system can serve as an effective camera-free dynamic shape sensing solution for soft robotics and other applications in dark, confined, or privacy-sensitive environments.

I. INTRODUCTION

Accurate soft-robotic 3D reconstruction plays a crucial role when soft robots operate in complex, constrained, and safety-sensitive environments. Tasks such as navigation in minimally invasive instruments, manipulation within endoscopic scenes, or interaction in confined spaces demand precise geometry recovery of deformable structures and surrounding environments [1-3]. Unlike rigid robots, soft robots introduce additional challenges—continuous deformation, occlusion, and limited sensing access—that require reconstruction methods beyond conventional camera-based pipelines [4-10]. While camera-centric approaches like multi-view and neural surface methods (e.g., MUST3R and Neuralangelo) achieve high-fidelity geometry from images [4, 11], and splatting representations such as 3D Gaussian Splatting (3DGS), 4DGS, and deformable Gaussians enable real-time rendering and reconstruction [6, 12, 13], these vision-centric pipelines break down under the very conditions common to soft robotics—darkness, occlusion, and restricted line-of-sight [1]. These realities motivate the pursuit of camera-free alternatives for soft-robotic 3D reconstruction.

Camera-free sensing has progressed but remains limited. Wearable gloves and flexible e-skins estimate joint angles, contacts, or distributed strain [7, 8, 14], yet typically yield

sparse/local measurements rather than dense, globally consistent 3D surfaces. Electrical Impedance Tomography (EIT) has also been explored for soft/continuum robots [9, 10, 15, 16], but its inverse problem, electrode-layout specificity, and computational burden limit spatial resolution and real-time performance. Many pipelines still depend on external markers or vision for ground truth during development. Meanwhile, dynamic neural rendering shows that high-fidelity, framewise geometry can be recovered from multi-view RGB with known poses [5, 12]. Sparse-control and cage-style modules efficiently drive high-resolution primitives (meshes, point sets, Gaussians) while preserving global coherence [17, 18], and geometry-aware/trimming strategies further improve surface fidelity [19, 20].

In this paper, we tackle two challenges that arise for soft-robotic 3D reconstruction without cameras: (i) learning an expressive deformation representation from signals that are orders of magnitude sparser than camera observations; and (ii) achieving physically consistent rendering at interactive rate. We propose a **camera-free, unified, and scalable** deformation sensing and rendering system. Starting from the robot’s nominal STL mesh, we deploy a **flexible, tileable piezoresistive array** that measures a 10×10 resistance grid on a $3 \text{ cm} \times 3 \text{ cm}$ patch, yielding rich local deformation signals under bend/twist [8]. For supervision, we use a **stop-motion** protocol: scan key static poses, render dense multi-view images with accurate poses via Blender’s virtual cameras, and densify time via video frame interpolation [21]; a **4DGS** pipeline then produces temporally trackable Gaussian models for every frame [12]. Rather than regressing motions for millions of Gaussians directly, a graph-based predictor estimates low-dimensional control displacements that are propagated to all Gaussians via local weighting. Canonicalization per sensed region and re-binding of control-Gaussian weights enable **zero-shot transfer** across objects, and sensor tiling scales coverage naturally. Finally, a **real-time Gaussian-splatting renderer** converts predictions into photorealistic RGB views, with lightweight temporal smoothing for stability [6, 19, 20].

We validate the system on a soft robot undergoing bending and twisting. The result is a sensor-driven digital twin that supports real-time monitoring and control of soft-body deformations in camera-denied conditions, aligning with recent trends in tactile learning and embedded perception for robotics [1, 3].

This paper makes the following contributions: (1) a camera-free, end-to-end pipeline for sensor-driven 3D defor-

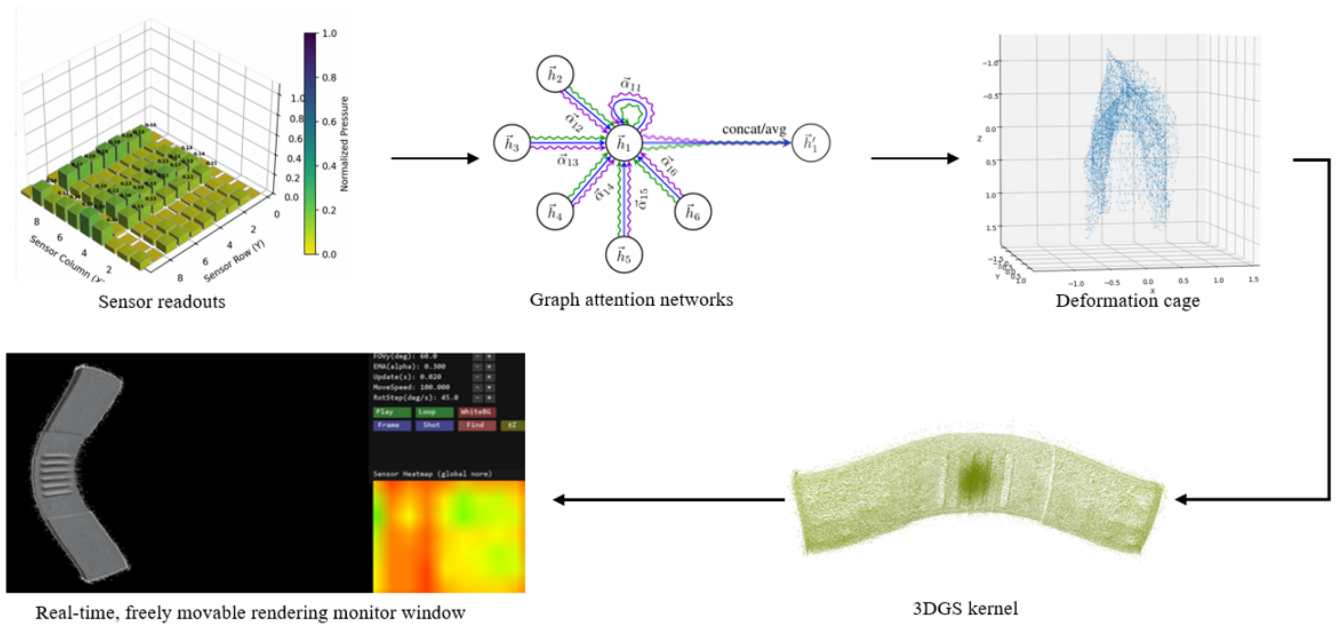


Fig. 1. brief overview of our pipeline

mation reconstruction and photorealistic visualization from a nominal STL/OBJ, deployable without cameras at test time; (2) a practical data-collection protocol that converts key-pose 3D scans into dense multi-view supervision and temporally rich training data via Blender rendering and frame interpolation; (3) a reproducible baseline and recipe for soft-robotics sensor-to-3D reconstruction (preprocessing, canonical coordinates, sparse-to-dense propagation, real-time rendering) with experiments showing $< 5\%$ mean 3D displacement error and ~ 30 FPS.

II. RELATED WORK

A. Sensor and Vision-Based Deformation Reconstruction

Capturing the 3D geometry of deformable soft robots has been explored through both sensor-based and vision-based approaches. Early sensor systems employed sparse resistive or capacitive threads in gloves [7, 22–24] or liquid-metal traces embedded in actuators [25, 26], which primarily drove proxy skeletons rather than detailed surfaces. More recent distributed e-skins—based on fiber Bragg gratings, carbon nanotube composites, or strain arrays [8, 27, 28]—have improved strain mapping but often lack spatial continuity and photorealism. Electrical Impedance Tomography (EIT) provides denser conductivity fields [9, 16, 29], yet remains layout-specific, calibration-heavy, and computationally demanding, typically producing maps that are difficult to render in real time [30, 31]. Recent learning-enhanced skins achieve millimeter-scale accuracy [14], though most pipelines still rely on external ground-truth systems.

Meanwhile, vision-based reconstruction has leveraged multi-view stereo and dynamic SLAM to recover time-varying meshes or point clouds, but these methods require strong texture cues and struggle under occlusion or real-time constraints. Neural scene representations such as NeRF and

its dynamic variants offer higher-fidelity modeling of non-rigid motions, yet demand curated multi-view data and precise calibration, which limits real-world deployment. Even with particle-grid dynamics [32] and 4D neural field interpolation [33], scalability and efficiency for high-resolution, real-time robotics remain open challenges.

B. 4D Gaussian Splatting

4D Gaussian Splatting (4DGS) makes dynamic reconstruction explicit by optimizing millions of anisotropic 3D Gaussians (with position, covariance, color, and opacity) over time. Compared to NeRF, this representation achieves high visual fidelity with real-time novel-view rendering, while also providing temporally trackable primitives that naturally align with surface deformation [6, 12, 34]. These properties make 4DGS an excellent supervision signal for learning sensor-to-shape mappings.

Beyond baseline 4DGS, numerous extensions enhance either geometry fidelity or physical grounding. TrimGS introduces contribution-based Gaussian trimming to improve surface accuracy [20], while GeoGaussian [19] and GausSurf [35] enforce geometry consistency with explicit priors. SC-GS [36] and NeuroGauss4D-PCI [33] reduce motion representation complexity by introducing sparse control structures. Other works integrate physical priors—such as PhysGaussian [37] and Physically Embodied GS [38]—to capture not just appearance but also material behavior. Despite these advances, 4DGS inherits stringent data requirements (multi-view, temporally dense RGB with accurate poses), and physically constrained Gaussian formulations, while insightful, remain computationally heavy for real-time inference.

C. Cage-Based Deformation and GNNs

Directly predicting motions for millions of elements (mesh vertices, point samples, or Gaussians) is computationally

prohibitive. Cage-based deformation provides an elegant solution: a low-resolution “cage” of control vertices drives the deformation of high-resolution targets. Classical approaches used harmonic or mean-value coordinates to propagate cage motions to meshes, preserving shape coherence at low cost. Recent methods extend this paradigm to Gaussian splatting, binding each Gaussian to a small set of cage nodes.

Such control–representation decoupling is particularly well-suited to sensor-driven reconstruction: instead of learning dense field trajectories, a model only needs to infer low-dimensional cage dynamics. Modern works further augment cages with learnable priors—e.g., Bézier patch cages [39], cage-based deformation for Gaussian splats [18, 40], and sparse control–Gaussian bindings [36]—demonstrating both efficiency and editability. In our setting, we treat the cage as a graph and apply a graph-attention network to model spatial relations among nodes, enriched with Fourier positional encodings and temporal embeddings. Combined with canonicalization and cage-to-Gaussian re-binding, this yields real-time inference, physically plausible outputs, and zero-shot transfer across novel objects.

III. METHOD

Our method comprises: (A) high-fidelity, temporally trackable Gaussian data acquisition; (B) flexible sensor hardware and data capture; (C) scalable real-time cage-node deformation prediction; and (D) real-time Gaussian rendering. A brief pipeline is in Fig. 1.

A. High-Fidelity, Trackable Dynamic Gaussian Data Acquisition

To obtain dense, time-consistent supervision, we build a high-fidelity 4D Gaussian dataset in three steps.

1) *Multi-View Keyframe Capture*: For each motion (bend, twist, press), we capture ~ 10 key poses with a structured-light 3D scanner and render each mesh from 100 virtual viewpoints in Blender, producing a sparse multi-view sequence. A stop-motion protocol ensures stable alignment: objects are rigidly clamped, backgrounds kept textureless, and camera parameters are exported consistently with the 4DGS convention.

2) *Frame Interpolation*: We densify time using a pre-trained interpolation network (RIFE [41]), synthesizing $\times 8\text{--}\times 16$ intermediate frames per view. The result is ~ 100 views $\times 160$ frames per motion, providing smooth temporal coverage without excessive dataset size.

3) *4D Gaussian Splatting (4DGS)*: We then train a 4DGS model on the multi-view videos and poses, yielding temporally corresponded Gaussians that track surface deformation across frames. Training follows the standard recipe [12], with $\text{RGB}+\ell_2/\text{SSIM}$ losses and weak regularization (opacity, covariance, temporal smoothness) to stabilize trajectories. Typical runs converge with $5 \times 10^5\text{--}2 \times 10^6$ splats depending on surface complexity.

B. Flexible Array Sensor Design and Data Collection

We fabricate a 3×3 cm tileable patch consisting of three layers by following the previous literature [3, 42]: (i) silver-plated nylon electrodes forming 10×10 orthogonal strips (~ 2 mm width, ~ 1 mm spacing); (ii) a commercial Velostat piezoresistive sheet in the middle; and (iii) TPU film encapsulation (Fig. 1(a)). Two 16-channel multiplexers (CD74HC4067) scan rows/columns in a voltage-divider circuit, and an ESP32 digitizes 100 channels (12-bit, 200–500 Hz) into a 10×10 resistance image per frame. A one-time two-point calibration maps raw voltages to resistances with per-channel baseline R_0 . During capture, we apply a low-pass IIR filter (8–12 Hz) and a 3×3 median filter for temporal and spatial denoising. For tiled setups, patches share a sync line and use unique I²C addresses, enabling time-stamped concatenation into a single $H \times W$ resistance image. Each patch is rigidly aligned to the local cage surface by Procrustes fitting of four fiducial corners. Similar tactile embedding strategies have been shown to facilitate generalization across manipulation tasks.

C. Scalable Real-Time Cage Node Deformation Prediction

We predict low-dimensional cage motions from sparse sensor inputs, then propagate to all Gaussians.

1) *Cage Representation and Gaussian Binding*: Each sensor region is enclosed by a regular-grid cage (normalized canonical frame aligned with the local surface normal). During preprocessing, each Gaussian center p_j binds to its k nearest cage nodes c_i via fixed interpolation weights w_{ij} (e.g., mean value coordinates), producing a sparse mapping.

a) *IDW variant (implementation)*: For inverse-distance weighting with k -NN $\mathcal{N}_k(j)$,

$$\phi_{ij} \triangleq \frac{1}{\|p_j - c_i\| + \varepsilon}, \quad Z_j \triangleq \sum_{i' \in \mathcal{N}_k(j)} \phi_{i'j},$$

$$w_{ij} = \frac{\phi_{ij}}{Z_j}, \quad \sum_{i \in \mathcal{N}_k(j)} w_{ij} = 1.$$

Runtime propagation from cage displacements Δc_i to Gaussians is

$$\Delta p_j = \sum_{i \in \mathcal{N}_k(j)} w_{ij} \Delta c_i.$$

2) *Graph Neural Network-Based Cage Deformer*: A CNN encoder processes the $H \times W$ sensor grid into a feature vector; a time encoder embeds normalized frame index; the concatenated feature conditions a GAT-based deformer over cage nodes (graph edges from cage connectivity), predicting per-node Δc_i (Fig. 2)

a) *Ground-truth cage labels*: Given 4DGS-derived Δp_j^{gt} and fixed w_{ij} , we solve a per-frame least squares:

$$\min_{\{\Delta c_i^{\text{gt}}\}} \sum_j \left\| \Delta p_j^{\text{gt}} - \sum_{i \in \mathcal{N}_k(j)} w_{ij} \Delta c_i^{\text{gt}} \right\|_2^2.$$

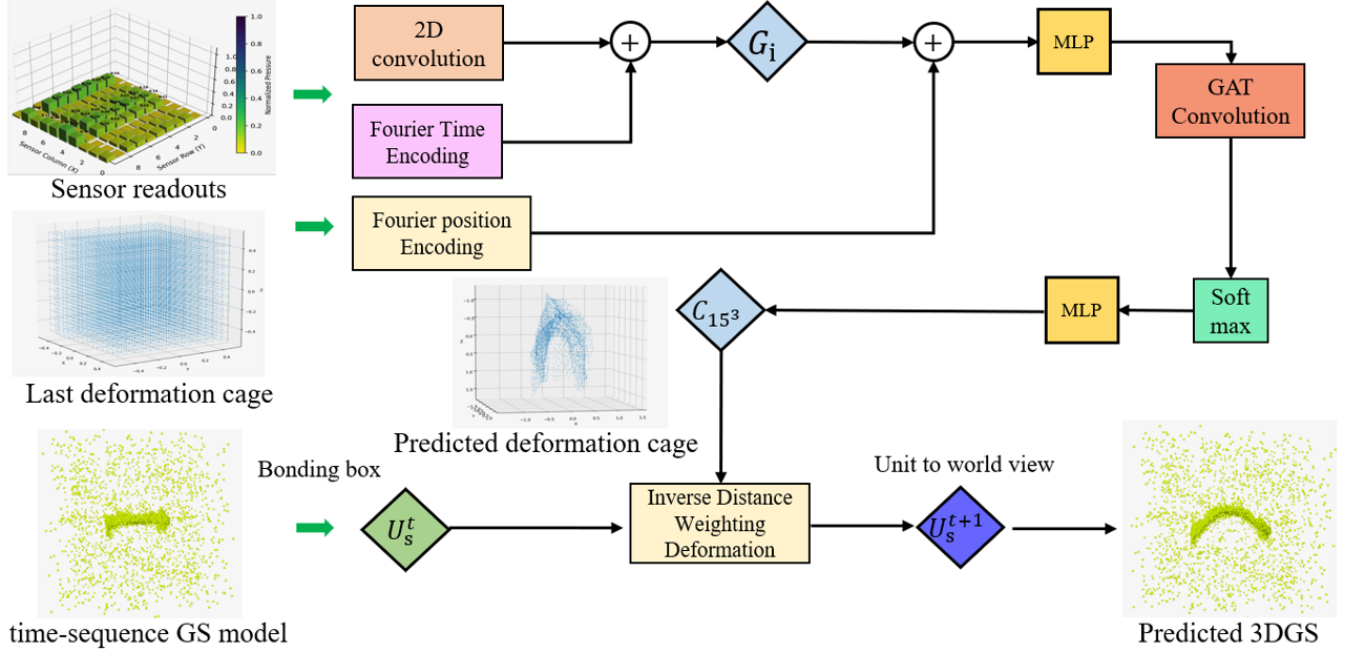


Fig. 2. The architecture of GAT cage-based deformation networks.

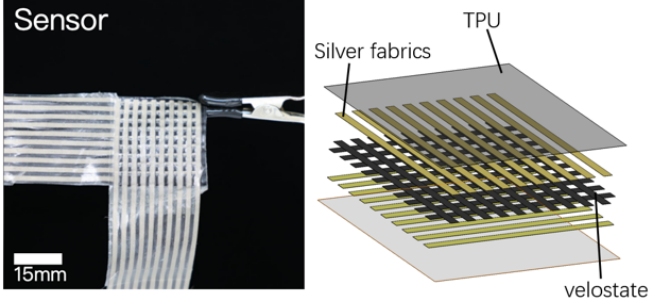


Fig. 3. Prototype sensor and corresponding exploded-view illustration.

b) Training objective.: With pairs $(R(t), \Delta c_i^{\text{gt}}(t))$, we minimize

$$\mathcal{L} = \frac{1}{N_c} \sum_{i=1}^{N_c} \|\Delta c_i^{\text{pred}}(t) - \Delta c_i^{\text{gt}}(t)\|_2^2 + \lambda \|\Delta c_i^{\text{pred}}(t) - \Delta c_i^{\text{pred}}(t-1)\|_2^2.$$

At inference, we optionally apply EMA smoothing

$$\Delta c_i^{\text{EMA}}(t) = \alpha \Delta c_i^{\text{EMA}}(t-1) + (1-\alpha) \Delta c_i(t), \quad 0 \leq \alpha < 1.$$

Updated Gaussian centers are

$$\hat{p}_j = p_j + \Delta p_j.$$

c) Network architecture and canonicalization: The sensor encoder is a lightweight 2D CNN. Time is encoded with six sinusoidal bands and concatenated with the sensor feature. Cage nodes are embedded by Fourier features of their canonical coordinates; the deformer comprises two GAT layers (4 heads, 256 hidden) and one GraphConv layer (256

hidden) with dropout 0.1, projecting to 3D node offsets via an MLP. We build the cage graph with 6-neighborhood connectivity on the regular grid. Canonicalization maps each sensor ROI to a unit cube via its oriented bounding box (origin at ROI centroid, axes from PCA, scale by diagonal length), which enables zero-shot transfer and weight reuse across surfaces. We precompute the sparse cage to Gaussian weights as a CSR matrix and keep it on the GPU; at runtime, propagation is a single sparse–dense matvec, which dominates neither compute nor memory.

D. Real-Time Rendering and Interaction

We render deformed Gaussians with a 3DGS-style rasterizer [6]. Given camera projection Π , each Gaussian projects to (μ'_j, Σ'_j) , and contributes

$$G_j(\mathbf{x}) = \alpha_j \exp\left(-\frac{1}{2}(\mathbf{x} - \mu'_j)^\top (\Sigma'_j)^{-1}(\mathbf{x} - \mu'_j)\right).$$

Alpha compositing yields

$$\mathbf{C}(\mathbf{x}) = \sum_{j=1}^{N_{\text{Gauss}}} T_j(\mathbf{x}) \alpha_j(\mathbf{x}) \mathbf{c}_j, \\ T_j(\mathbf{x}) = \prod_{k=1}^{j-1} [1 - \alpha_k(\mathbf{x})].$$

Since $(\Sigma_j, \alpha_j, \mathbf{c}_j)$ are fixed from 4DGS, only centers update per frame, enabling interactive rates. A simple UI controls exposure, FOV, and camera motion; EMA smoothing further stabilizes predictions.

a) System settings and UI: We target 1280×800 to 1600×1000 render resolutions with per-splat anisotropic footprints and front-to-back alpha compositing. Depth sorting is done per tile to reduce overdraw; near/far culling and

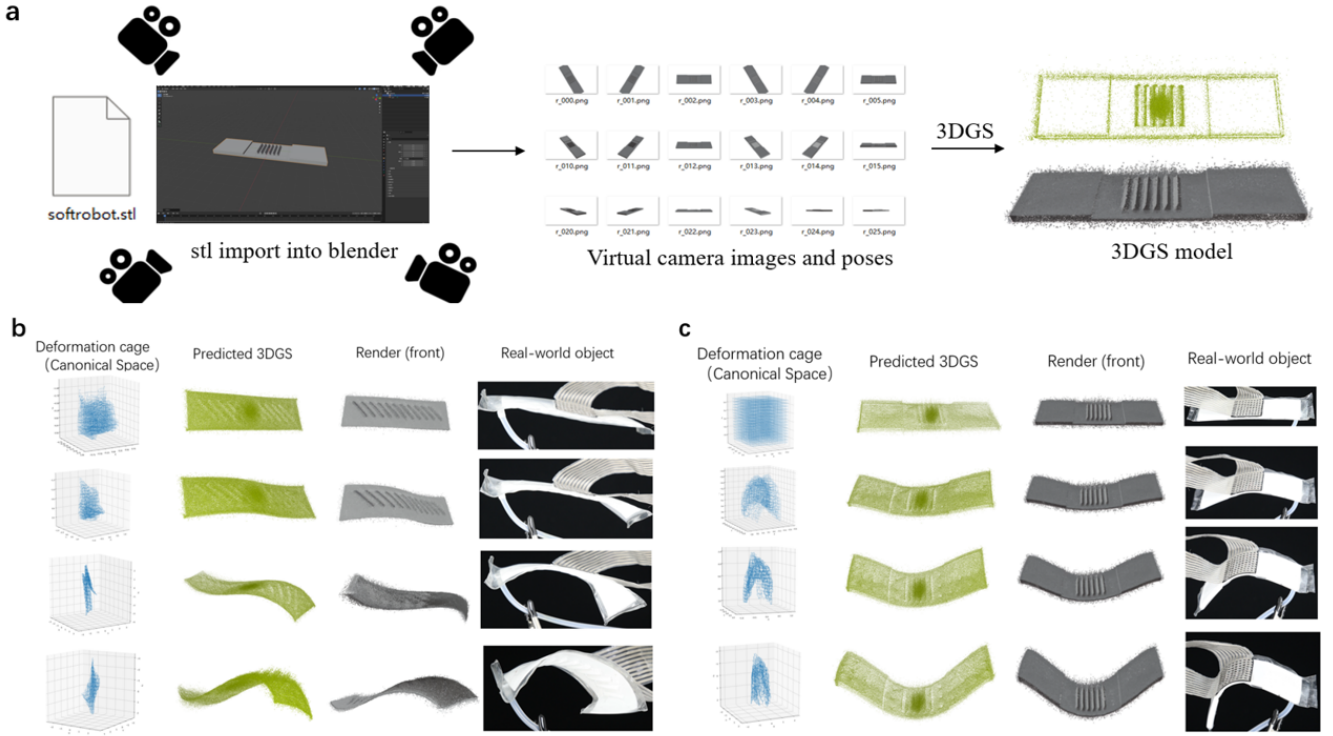


Fig. 4. **Experiment results analysis** (a) Pipeline to create 3DGS model from stl file. (b) Twisting experiment results. (c) Bending experiment results.

frustum clipping are enabled by default. The UI exposes camera orbit/dolly, FOV, exposure, and EMA smoothing coefficient α ; we found $\alpha \in [0.6, 0.8]$ removes jitter without noticeable lag. All parameters can be changed live, enabling teleoperation and quick what-if analysis during deployment.

IV. EXPERIMENTS

A. Experimental Setup and Dataset

We evaluate our approach on two deformable soft robots. The robot is a silicone structure that can undergo bending and twisting motions. We instrumented the robot with our sensor array and created deformation sequences for each motion type:

- **Data Acquisition:** Using an *EinScan SE* structured-light 3D scanner, we obtained high-resolution textured meshes for 10 key poses of each motion. These meshes were used to generate multi-view images as described in Section III-A. We collected 100 views for each key pose, then performed $16 \times$ frame interpolation, yielding a total of ~ 160 frames per view. In total, our image dataset for each motion comprised $\sim 16,000$ images ($100 \text{ views} \times 160 \text{ frames}$). We split each set into 80% training, 10% validation, and 10% test frames (ensuring that entire frames, not individual views, were separated to test generalization to unseen time steps). The images and corresponding 4D-GS outputs were used to supervise the deformation model.(Fig.5)
- **Sensor Data:** For each motion sequence, we attached the sensor to regions where the deformation is promi-



Fig. 5. **bending and twisting data acquisition**

nent. During the motion sequences (performed quasi-statically for scanning), we recorded the sensor’s resistance matrix at each frame. After data collection, we had synchronized sequences of sensor readings and ground-truth Gaussian deformations for each region and motion. All data was normalized: sensor resistances were normalized per-channel (subtracting the resting value and dividing by the range), and cage/Gaussian coordinates for each region were normalized to that region’s local frame as explained earlier.

a) Acquisition specifics and synchronization.: The Ein-Scan SE is operated at its highest precision setting (reported point spacing 0.05–0.1 mm). For each motion type we capture 10 key poses over the full actuation range (uniform steps for bend/twist; pressure steps for press), render 100 viewpoints per pose, and interpolate by $\times 16$, yielding $\sim 1.6 \times 10^4$ images per motion. Sensor readouts are recorded at 250 Hz and downsampled to match the 4DGS frame timestamps by nearest-neighbor alignment; a trigger pulse from the ESP32 marks the start of each rendered sequence to ensure consistent indexing. We randomize sensor placement across trials (convex/concave faces for bend; lateral faces for twist) to improve coverage and reduce bias. Before training, all resistance maps are normalized per-channel by $(R - R_0)/(\max R - \min R)$ over the entire session, and the corresponding cage and Gaussian coordinates are transformed to the canonical frame described in Sec. III.

B. Implementation and Training Details

The model was implemented in PyTorch and PyTorch Geometric. Each sensor-attached region is represented by a cage of $N_c = 3375$ control points arranged in a $15 \times 15 \times 15$ grid. We trained both motion-specific models and a unified multi-motion model for 100 epochs with a batch size of 8 on a single NVIDIA RTX 3070Ti GPU. The Adam optimizer was used with an initial learning rate of 1×10^{-3} , decayed to 1×10^{-4} after 50 epochs. Validation loss was monitored throughout, and early stopping was applied if no improvement was observed for 10 epochs. The checkpoint with the lowest validation loss was used for evaluation.

a) Architectural and optimization details.: We implement the sensor encoder and deformer in PyTorch Geometric with mixed precision (FP16) to reduce memory. Batches consist of 8 frames sampled from random time windows (length 16) to encourage temporal consistency. We use Adam with weight decay 1×10^{-5} , gradient clipping at 1.0, and cosine learning-rate decay; dropout 0.1 is applied in GAT layers. Data augmentation includes per-frame multiplicative gain jitter on the resistance map ($\pm 5\%$), Gaussian noise ($\sigma = 0.01$), and random masking of up to 2% sensor pixels to mimic transient contact loss. The temporal smoothness weight λ in the loss is set to 1×10^{-3} ; the EMA smoothing coefficient α at inference is 0.7 unless stated otherwise. For propagation we use $k = 2048$ nearest cage nodes and $\varepsilon = 10^{-6}$ in Eq. (III-C.1.a). A typical training run (100 epochs) takes ~ 6 – 8 hours on an RTX 3070 Ti; peak GPU memory is ~ 6 – 7 GB with $N_c=3375$ and 100k splats loaded for supervision.

V. RESULTS

A. Cage Deformation Prediction and Runtime

To verify whether our model captures both the sensor input and the underlying physical relations reflected in cage nodes, we train it only on partial stages of motion. Specifically, for *bend*, we use sensor data from the early and late phases (before and after maximal bending), while leaving most intermediate states unobserved. The model successfully

infers the entire bending sequence, producing smooth and physically plausible deformations across the full trajectory (Fig. 4(b)(c)). We further conduct a quantitative analysis of bending accuracy: by comparing the bending angles measured from ground-truth camera images and our rendered reconstructions at the same timestamps and viewpoints, the angular error remains below 4.7° (Fig. 7).

Runtime efficiency remains high. On a single RTX 3070 Ti, inference reaches ~ 30 FPS with ~ 30 k Gaussians (coarse setting), and ~ 5 FPS with ~ 100 k Gaussians (high-precision setting), highlighting the efficiency of controlling a large Gaussian set via a small cage. Most runtime is spent in the sensor encoder and Gaussian rendering; the GNN deformer is lightweight due to the modest cage size. Our temporal smoothing and boundary-aware propagation maintain continuity between deformed and static regions, avoiding visible cracks at region interfaces.

B. Zero-Shot Camera-Free Reconstruction on New Objects

By combining region *bounding boxes* with canonical normalization, our model deforms regions after mapping them into a normalized space, enabling generalization to previously unseen surfaces and objects. Users define the predicted deformation region according to the physical size of the flexible sensor, and the system then applies the trained cage deformer for inference. For unseen objects without any camera input at test time, we import their STL/OBJ into Blender and use our virtual-camera pipeline to generate a static Gaussian model (Fig. 4a). After specifying the sensor footprint, the trained deformer yields real-time shape reconstructions. Gaussian Splatting subsequently produces photorealistic RGB renderings with correct occlusion and perspective, which help interpret local surface changes. For instance, in the wallet experiment, we tested a 10×10 sensor. Although the wallet does not appear in the training set, the model still predicts plausible bending and pressing deformations when the ROI is selected on the initial Gaussian model, supporting real-time zero-shot rendering (Fig. 6).

VI. DISCUSSION

Advantages: The proposed system achieves locally dense, high-resolution shape sensing without external cameras (except for data collection phases) or markers, allowing soft robots to reconstruct their shapes in dark, occluded, or privacy-sensitive settings. Leveraging 4D Gaussian Splatting enables both accurate geometry reconstruction and photorealistic rendering, creating a “live mirror” of the soft robot for monitoring or teleoperation. The cage-based GNN balances physical plausibility and flexibility: its structure enforces spatial coherence, while attention weights capture complex non-linear deformations. The model is lightweight and runs in real time on a single GPU. In addition, the modular design of the sensor array allows tiling multiple patches to scale up coverage, while keeping wiring and calibration manageable. The use of canonicalization enables zero-shot transfer across different objects and regions, reducing retraining cost. EMA



Fig. 6. **Results from more complicated objects** Users can define the inference region according to the sensor’s size and set the positive normal direction. Here we show the rendered result on a wallet, which was not included in the training set, by attaching the sensor on top and pressing it.

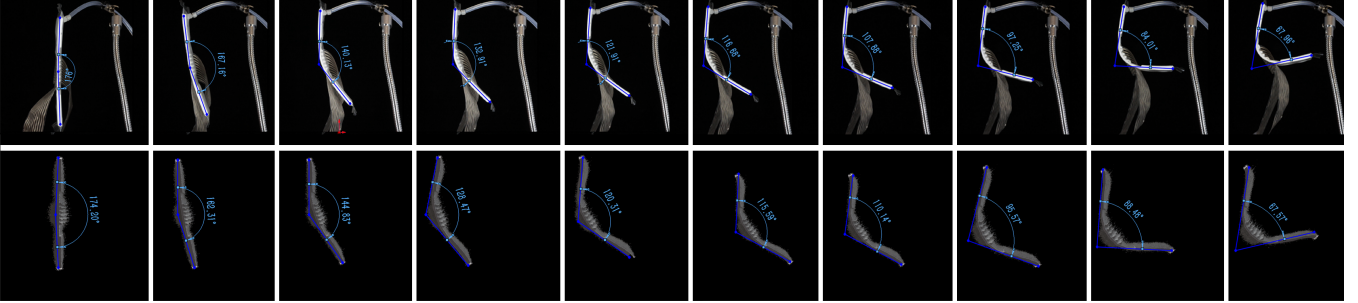


Fig. 7. **Quantitative analysis of bending accuracy.** The top row shows the real captured bending scenes, while the bottom row presents the results rendered in real time from the sensor.

smoothing and temporal priors stabilize predictions, producing visually consistent trajectories suitable for deployment. The system integrates sensing, learning, and rendering in a unified pipeline, which simplifies data synchronization and minimizes latency. Finally, the framework is extensible: it can incorporate higher-density sensors, inertial signals, or physics-based priors without changing the overall architecture.

Limitations: Each new object requires an offline scan and 4D-GS preprocessing to build its Gaussian model. The current sensor (100 points in a 3×3 cm area) may miss highly localized deformations smaller than the grid spacing. Our design assumes quasi-static or slow motions; high-speed dynamics are not explicitly modeled. Moreover, during reconstruction we do not explicitly constrain Gaussian kernel shape or opacity evolution, which can introduce fidelity loss in regions with strong shading or topology changes. From an algorithmic perspective, the learned cage-to-Gaussian mapping remains sensitive to out-of-distribution inputs—e.g., extreme motions or unseen material properties—and may accumulate prediction drift without re-calibration. On the hardware side, the flexible sensor patch cannot conform reliably under very large surface-area changes, limiting its applicability to scenarios with severe stretching or folding.

VII. CONCLUSION

We presented a camera-free, real-time deformation reconstruction system using a flexible sensor array and a cage-based GNN, translating sparse resistance data into detailed 3D Gaussian models. Experiments on a soft robotic specimen demonstrated high accuracy, plausible dynamics, and photorealistic rendering. The cage representation was key to tractable and generalizable inference. Future work will

focus on simplifying deployment, increasing sensing resolution, expanding motion coverage, and exploring applications from structural monitoring to animating virtual avatars. This approach offers a practical step toward real-time camera-free “digital twins” for soft structures in robotics, healthcare.

REFERENCES

- [1] Y. Huang, B. Cui, L. Bai, Z. Guo, M. Xu, M. Islam, and H. Ren. Endo-4DGS: Endoscopic Monocular Scene Reconstruction with 4D Gaussian Splatting. [Online]. Available: <http://arxiv.org/abs/2401.16416>
- [2] Electrical Impedance Tomographic Shape Sensing for Soft Robots - ADS. [Online]. Available: <https://ui.adsabs.harvard.edu/abs/2023IRAL....8.1555X>
- [3] Y. Luo, M. Wonsick, J. Hodgins, and B. Okorn, “Tactile Embeddings for Multi-Task Learning,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, pp. 3348–3355. [Online]. Available: <https://ieeexplore.ieee.org/document/10611419/>
- [4] Z. Li, T. Müller, A. Evans, R. H. Taylor, M. Unberath, M.-Y. Liu, and C.-H. Lin. Neuralangelo: High-Fidelity Neural Surface Reconstruction. [Online]. Available: <http://arxiv.org/abs/2306.03092>
- [5] L. Song, A. Chen, Z. Li, Z. Chen, L. Chen, J. Yuan, Y. Xu, and A. Geiger, “NeRFPlayer: A Streamable Dynamic Scene Representation with Decomposed Neural Radiance Fields,” vol. 29, no. 5, pp. 2732–2742. [Online]. Available: <https://doi.org/10.1109/TVCG.2023.3247082>
- [6] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. [Online]. Available: <http://arxiv.org/abs/2308.04079>
- [7] M. Park, T. Park, S. Park, S. J. Yoon, S. H. Koo, and Y.-L. Park, “Stretchable glove for accurate and robust hand pose reconstruction based on comprehensive motion data,” vol. 15, no. 1, p. 5821. [Online]. Available: <https://www.nature.com/articles/s41467-024-50101-w>
- [8] S. Mei, H. Yi, J. Zhao, Y. Xu, L. Shi, Y. Qin, Y. Jiang, J. Guo, Z. Li, and L. Wu, “High-density, highly sensitive sensor array of spiky carbon nanospheres for strain field mapping,” vol. 15, no. 1, p. 3752. [Online]. Available: <https://www.nature.com/articles/s41467-024-47283-8>
- [9] H. Dong, X. Wu, D. Hu, Z. Liu, F. Giorgio-Serchi, and Y. Yang, “Learning-enhanced electronic skin for tactile sensing on deformable

- surface based on electrical impedance tomography,” vol. 74, pp. 1–9. [Online]. Available: <http://arxiv.org/abs/2504.05987>
- [10] Z. Jiang, Z. Xu, M. Li, H. Zeng, F. Gong, and Y. Tang, “Electrical Impedance Tomography-Based Electronic Skin for Multi-Touch Tactile Sensing Using Hydrogel Material and FISTA Algorithm,” vol. 24, no. 18, p. 5985. [Online]. Available: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11435466/>
- [11] Y. Cabon, L. Stoffl, L. Antsfeld, G. Csurka, B. Chidlovskii, J. Revaud, and V. Leroy. MUST3R: Multi-view Network for Stereo 3D Reconstruction. [Online]. Available: <http://arxiv.org/abs/2503.01661>
- [12] G. Wu, T. Yi, J. Fang, L. Xie, X. Zhang, W. Wei, W. Liu, Q. Tian, and X. Wang. 4D Gaussian Splatting for Real-Time Dynamic Scene Rendering. [Online]. Available: <http://arxiv.org/abs/2310.08528>
- [13] Z. Yang, X. Gao, W. Zhou, S. Jiao, Y. Zhang, and X. Jin. Deformable 3D Gaussians for High-Fidelity Monocular Dynamic Scene Reconstruction. [Online]. Available: <http://arxiv.org/abs/2309.13101>
- [14] D. Hu, F. Giorgio-Serchi, S. Zhang, and Y. Yang, “Stretchable e-skin and transformer enable high-resolution morphological reconstruction for soft robots,” vol. 5, no. 3, pp. 261–272. [Online]. Available: <https://www.nature.com/articles/s42256-023-00622-8>
- [15] W. Xin, F. Zhu, P. Wang, Z. Xie, Z. Tang, and C. Laschi, “Electrical Impedance Tomographic Shape Sensing for Soft Robots,” vol. 8, pp. 1555–1562. [Online]. Available: <https://ui.adsabs.harvard.edu/abs/2023IRAL.....8.1555X>
- [16] H. Chen, B. Himmel, B. Li, X. Wang, and M. Hoffmann. Multi-Touch and Bending Perception Using Electrical Impedance Tomography for Robotics. [Online]. Available: <http://arxiv.org/abs/2503.13048>
- [17] Y.-H. Huang, Y.-T. Sun, Z. Yang, X. Lyu, Y.-P. Cao, and X. Qi. SC-GS: Sparse-Controlled Gaussian Splatting for Editable Dynamic Scenes. [Online]. Available: <http://arxiv.org/abs/2312.14937>
- [18] J. Huang, S. Xu, H. Yu, and T.-Y. Lee. GSDeformer: Direct, Real-time and Extensible Cage-based Deformation for 3D Gaussian Splatting. [Online]. Available: <http://arxiv.org/abs/2405.15491>
- [19] Y. Li, C. Lyu, Y. Di, G. Zhai, G. H. Lee, and F. Tombari. GeoGaussian: Geometry-aware Gaussian Splatting for Scene Rendering. [Online]. Available: <http://arxiv.org/abs/2403.11324>
- [20] L. Fan, Y. Yang, M. Li, H. Li, and Z. Zhang. Trim 3D Gaussian Splatting for Accurate Geometry Representation. [Online]. Available: <http://arxiv.org/abs/2406.07499>
- [21] Z. Li, Z.-L. Zhu, L.-H. Han, Q. Hou, C.-L. Guo, and M.-M. Cheng, “AMT: All-Pairs Multi-Field Transforms for Efficient Frame Interpolation,” in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, pp. 9801–9810. [Online]. Available: <https://ieeexplore.ieee.org/document/10205210/>
- [22] J. DelPreto, J. Hughes, M. D’Aria, M. De Fazio, and D. Rus, “A wearable smart glove and its application of pose and gesture detection to sign language classification,” vol. 7, no. 4, pp. 10589–10596. [Online]. Available: <https://ieeexplore.ieee.org/document/9830849/>
- [23] S. Sundaram, P. Kellnhofer, Y. Li, J.-Y. Zhu, A. Torralba, and W. Matusik, “Learning the signatures of the human grasp using a scalable tactile glove,” vol. 569, no. 7758, pp. 698–702. [Online]. Available: <https://www.nature.com/articles/s41586-019-1234-z>
- [24] H. Dong, C. Wang, M. Jiang, F. Giorgio-Serchi, and Y. Yang, “Modular soft wearable glove for real-time gesture recognition and dynamic 3d shape reconstruction.” [Online]. Available: <http://arxiv.org/abs/2504.05983>
- [25] R. L. Truby, R. K. Katzschmann, J. A. Lewis, and D. Rus, “Soft robotic fingers with embedded ionogel sensors and discrete actuation modes for somatosensitive manipulation,” in *2019 2nd IEEE International Conference on Soft Robotics (RoboSoft)*. IEEE, pp. 322–329. [Online]. Available: <https://ieeexplore.ieee.org/document/8722722/>
- [26] R. L. Truby, C. D. Santina, and D. Rus, “Distributed proprioception of 3d configuration in soft, sensorized robots via deep learning,” vol. 5, no. 2, pp. 3299–3306. [Online]. Available: <https://ieeexplore.ieee.org/document/9013033/>
- [27] T. L. T. Lun, K. Wang, J. D. L. Ho, K.-H. Lee, K. Y. Sze, and K.-W. Kwok, “Real-time surface shape sensing for soft and flexible structures using fiber bragg gratings,” vol. 4, no. 2, pp. 1454–1461. [Online]. Available: <https://ieeexplore.ieee.org/document/8611383/>
- [28] D. Chen, Y. Cai, L. Cheng, S. Guo, T. Liu, S. Huang, H. Yu, Y. Wang, Z. Hu, and D. Gui, “Structure and function design of carbon nanotube-based flexible strain sensors and their application,” vol. 225, p. 113992. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0263224123015567>
- [29] D. Hardman, T. G. Thuruthel, and F. Iida, “Multimodal information structuring with single-layer soft skins and high-density electrical impedance tomography.”
- [30] E. Koch and A. Dietzel, “Surface reconstruction by means of a flexible sensor array,” vol. 267, pp. 293–300. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S092442471730938X>
- [31] Y. Zhao, C. K. Khaw, and Y. Wang, “Measuring a soft resistive strain sensor array by solving the resistor network inverse problem.” [Online]. Available: <http://arxiv.org/abs/2304.05828>
- [32] K. Zhang, B. Li, K. Hauser, and Y. Li, “Particle-grid neural dynamics for learning deformable object models from depth images.”
- [33] C. Jiang, D. Du, J. Liu, S. Zhu, Z. Liu, Z. Ma, Z. Liang, and J. Zhou, “NeuroGauss4d-PCI: 4d neural fields and gaussian deformation fields for point cloud interpolation.”
- [34] G. Chen and W. Wang, “A survey on 3d gaussian splatting.” [Online]. Available: <http://arxiv.org/abs/2401.03890>
- [35] J. Wang, Y. Liu, P. Wang, C. Lin, J. Hou, X. Li, T. Komura, and W. Wang, “GausSurf: Geometry-guided 3d gaussian splatting for surface reconstruction.” [Online]. Available: <http://arxiv.org/abs/2411.19454>
- [36] Y.-H. Huang, Y.-T. Sun, Z. Yang, X. Lyu, Y.-P. Cao, and X. Qi, “SC-GS: Sparse-controlled gaussian splatting for editable dynamic scenes,” in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, pp. 4220–4230. [Online]. Available: <https://ieeexplore.ieee.org/document/10657650/>
- [37] T. Xie, Z. Zong, Y. Qiu, X. Li, Y. Feng, Y. Yang, and C. Jiang, “PhysGaussian: Physics-integrated 3d gaussians for generative dynamics.” [Online]. Available: <http://arxiv.org/abs/2311.12198>
- [38] J. Abou-Chakra, K. Rana, and F. Dayoub, “Physically embodied gaussian splatting: A visually learnt and physically grounded 3d representation for robotics.”
- [39] D. Xiao and R. Chen, “Flexible 3d cage-based deformation via green coordinates on bézier patches,” in *Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers*, pp. 1–10. [Online]. Available: <http://arxiv.org/abs/2501.14068>
- [40] Y. Tong, R. Tian, X. Han, D. Liu, F. Yu, and Y. Zhang, “CAGE-GS: High-fidelity cage based 3d gaussian splatting deformation.” [Online]. Available: <http://arxiv.org/abs/2504.12800>
- [41] Z. Huang, T. Zhang, W. Heng, B. Shi, and S. Zhou, “Real-time intermediate flow estimation for video frame interpolation.” [Online]. Available: <http://arxiv.org/abs/2011.06294>
- [42] B. Huang, Y. Wang, X. Yang, Y. Luo, and Y. Li, “3d-ViTac: Learning fine-grained manipulation with visuo-tactile sensing.” [Online]. Available: <http://arxiv.org/abs/2410.24091>